Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Measurement and Information Systems

Wireless protocol testing in embedded environment

Master's Thesis

Author
Dániel Ábrahám

Advisor
dr. András Vörös

June 6, 2023

Contents

Kivonat **i**

Abstract **ii**

1 Introduction **1**

1.1 Problem definition 1

1.2 Practical considerations 1

1.3 Thesis Outline 1

1.4 Contributions 1

2 Wireless protocols **2**

2.1 ISM radio band 2

2.2 Bluetooth Low Energy 2

2.2.1 Introcution 2

2.2.2 Architecture 3

2.2.3 LE Controller 4

2.2.3.1 Physical layer 4

2.2.3.2 Link layer 4

2.2.4 Host Controller Interface 6

2.2.5 Host 6

2.2.5.1 Logical Link Control and Adaption Protocol 6

2.2.5.2 Attribute Protocol 7

2.2.5.3 Generic Access Profile 7

2.2.5.4 Generic Attribute Profile 7

2.3 IEEE 802.15.4 7

2.3.1 Physical layer (PHY) 8

2.3.2 Medium Access Control Layer (MAC) 8

2.4 Zigbee 8

2.4.1 Introduction 8

2.4.2 Network . . . 9
2.4.3 Architecture . . . 9
2.4.4 Network layer . . . 9
2.4.5 Application layer . . . 9
2.4.5.1 Application Support Sub-layer . . . 9
2.4.5.2 Application Framework . . . 10
2.4.5.3 Zigbee Device Object . . . 10
2.4.6 Cluster Library . . . 10
2.5 Thread/OpenThread . . . 10
2.5.1 Introduction . . . 10
2.5.2 Private Area Network . . . 10

3 Multi-Protocol **15**
3.1 Overview . . . 15
3.2 Device configuration . . . 16
3.3 Implementations . . . 16
3.3.1 Dynamic Multi-protocol Manager - Texas Instruments . . . 16
3.3.2 Dynamic Multiprotocol - Silicon Labs . . . 16
3.3.3 CPCd - Silicon Labs . . . 16
3.3.4 Multi-Protocol Service Layer - Nordic Semiconductor . . . 16

4 Software testing **17**
4.1 Overview . . . 17
4.2 Testing levels . . . 17
4.3 Test types . . . 18
4.4 Model-based test generation . . . 18
4.5 Runtime verification . . . 18
4.6 Bluetooth testing . . . 18
4.7 Zigbee testing . . . 18
4.8 Thread/Openthread testing . . . 18

Acknowledgements **19**

Bibliography **20**

HALLGATÓI NYILATKOZAT

Alulírott *Ábrahám Dániel*, szigorló hallgató kijelentem, hogy ezt a diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy autentikált felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2023. június 6.

Ábrahám Dániel
hallgató

Kivonat

Jelen dokumentum egy diplomaterv sablon, amely formai keretet ad a BME Villamosmérnöki és Informatikai Karán végző hallgatók által elkészítendő szakdolgozatnak és diplomatervnek. A sablon használata opcionális. Ez a sablon LaTeX alapú, a *TeXLive* TeX-implementációval és a PDF-LaTeX fordítóval működőképes.

Abstract

This document is a LaTeX-based skeleton for BSc/MSc theses of students at the Electrical Engineering and Informatics Faculty, Budapest University of Technology and Economics. The usage of this skeleton is optional. It has been tested with the *TeXLive* TeX implementation, and it requires the PDF-LaTeX compiler.

Chapter 1

Introduction

1.1 Problem definition

1.2 Practical considerations

1.3 Thesis Outline

This thesis has two parts: a literature review and a contribution. The literature review will consist of Chapters 2, 3, and 4.

In Chapter 2, Wireless protocols will be shown. Chapter 3 will introduce the concept of multiprotocol networking, and Chapter 4 will introduce some software and system testing techniques.

1.4 Contributions

Chapter 2

Wireless protocols

In this chapter, I will describe the Industrial, Scientific and Medical (ISM) 2,4 GHz radio band in Section 2.1. After that, I will demonstrate each wireless protocol starting with Bluetooth Low Energy in Section 2.2, Zigbee in Section 2.4, and finally Thread-/OpenThread in Section 2.5. Section 2.3 will provide an overview of the IEEE 802.15.4 radio specification, of which both Zigbee and Thread is based on.

2.1 ISM radio band

2.2 Bluetooth Low Energy

Bluetooth is a low-power, short-range wireless technology developed initially for replacing cables when connecting devices like mobile phones, headsets, and computers. It has since evolved into a wireless standard for connecting electronic devices to form Personal Area Networks (PANs) and ad hoc networks. [6]

Bluetooth is one of the most popular commodity radios for wireless devices. As a representative of the frequency hopping spread spectrum radios, it is a natural alternative to broadcast radios in the context of sensor networks. [17]

The Bluetooth SIG specifies three (3) different types of devices, hence three specifications. Bluetooth Classic consists of Basic Rate and Enhanced Data Rate capable devices. On the other hand, Bluetooth Low Energy needs Low Energy capabilities.

Since BLE and Bluetooth Classic has different controller layer, they are not compatible with each other. This thesis will focus mainly on Bluetooth Low Energy (BLE) due to its similarity in functionality and utilization to Zigbee and Thread/OpenThread.

The Bluetooth Special Interest Group (SIG) oversees the Bluetooth standard and has a membership of over 38,000 companies as of 2023. [2] The initial specification was made available in 1999. Since then, the SIG released five core specifications, 5.4 being the latest adoption in 2023. [3]

2.2.1 Introcution

The Bluetooth 4.0 Core Specification introduced Bluetooth Low Energy (BLE). It is tempting to represent itself as a smaller and highly optimized version of its bigger brother, classic Bluetooth. In reality, BLE has an entirely different lineage and design goals. Ini-

tially known as Wibree and created by Nokia, the Bluetooth SIG ultimately embraced the technology. The focus was to develop a radio standard with the lowest possible power consumption, specifically optimized for low cost, low bandwidth, low power, and low complexity. [19]

BLE uses frequency hopped spread spectrum (FHSS), just like Bluetooth Classic, to limitate transmission errors in the radio band. The physical layer uses a 2 MHz bandwidth. From Core Specification 5, BLE has a theoretical maximum throughput of 2 Mbps and a maximum output power of 100 mW (20 dBm).

2.2.2 Architecture

According to Townsend et al., the Bluetooth Low Energy device can be categorized into three primary blocks: controller, host, and application. Each block has one or more layers, as shown in Figure 2.1.

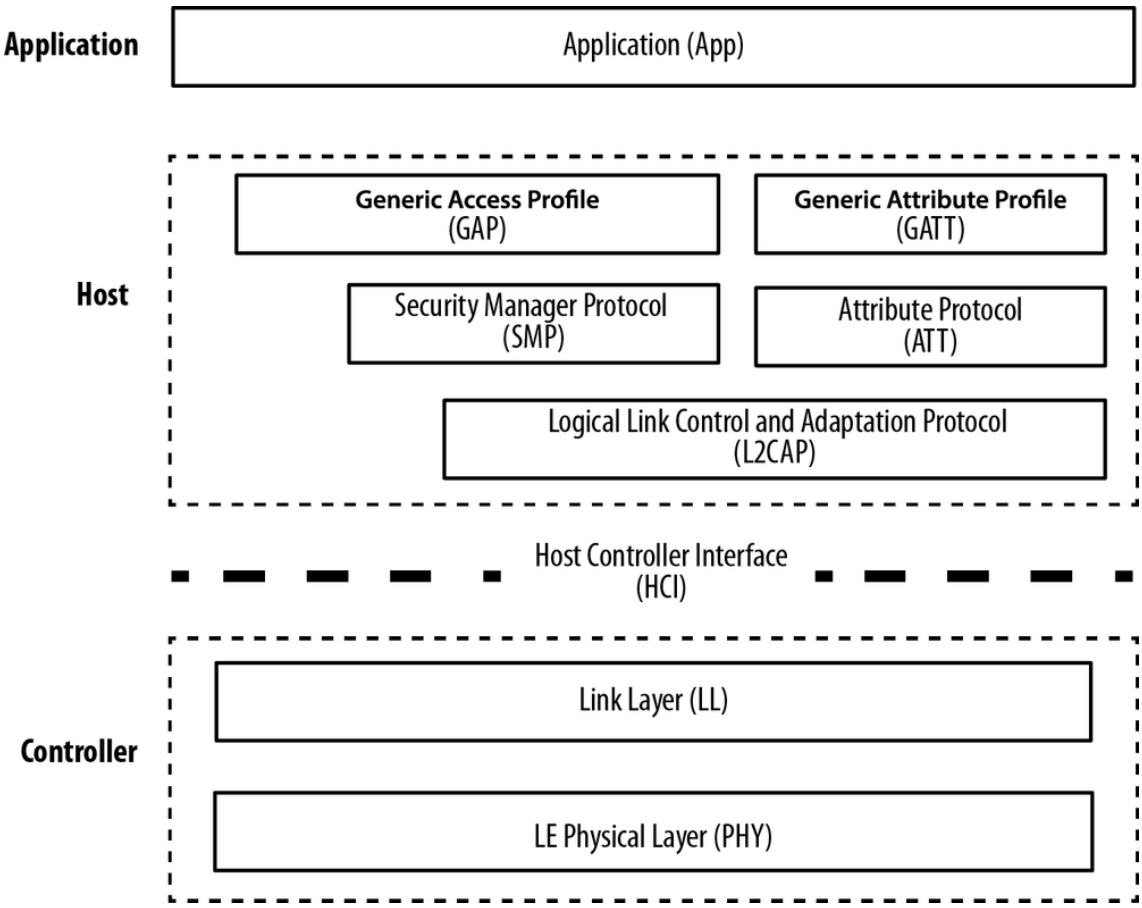

Figure 2.1: The BLE protocol stack. (Source: Figure 2-1 [19])

The layers defined in Figure 2.1 provide the required functionality to operate.

The Controller contains the Physical Layer (PHY) and the Link Layer (LL). These hold all the responsibility for operating the physical radio channel. Also, these layers are specialized by the LE standard keeping the Host layer more-or-less the same as Bluetooth Classic.

A device address and an address type identify a device. An address type indicates that an address can be either public or random. Both are 48 bits long. A device shall use at least one type of device address but can use both simultaneously.

In BLE, Generic Access Profile (GAP) defines specific roles for each device. These roles are: broadcaster, observer, peripheral, and central. A device may support multiple roles but only one at a given time. Each role specifies the requirements for the underlying Controller. **??** explains GAP in more detail.

2.2.3 LE Controller

2.2.3.1 Physical layer

The physical layer (PHY) contains all necessary analog communication circuitry. The radio uses the 2.4 GHz ISM band to communicate. This band is divided into 40 channels or subbands from 2.400 GHz to 2.4835 GHz, each subband using 2 MHz of bandwidth. (Yin et al., 2019) This division can be seen in Figure 2.2. There are three dedicated advertising subbands for connection setup and broadcast messages; the remaining 37 are known as general-purpose channels.

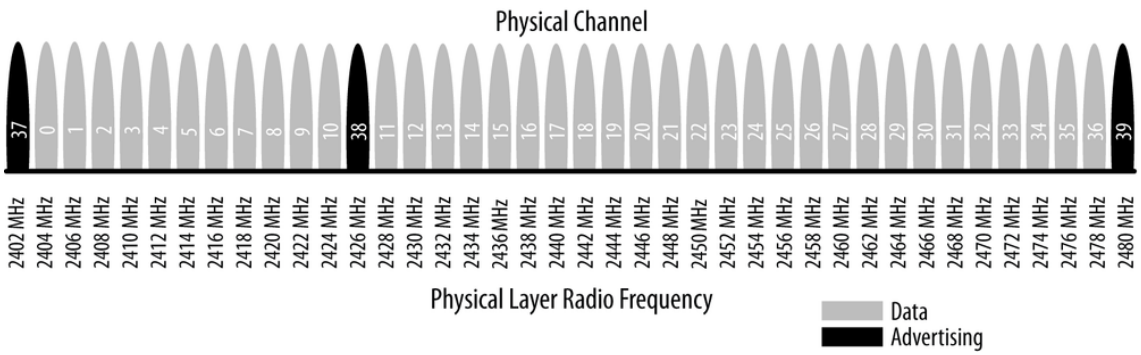

Figure 2.2: Channel allocation. (Source: Figure 2-2 [19])

The transmitter can use a maximum power of 100 mW. The LE Power Control Request can manage the power settings if a device supports this feature.

J. Yin et al. [23] showed that from Core Specification 5, a novel modulation scheme was introduced in the standard. This feature allowed radio modules to use up all 2 MHz of bandwidth, thus significantly increasing the maximum throughput.

2.2.3.2 Link layer

Part B of the Core Specification's volume six (Low Energy Controller volume) defines the Bluetooth Low Energy Link Layer's specification.

The Link Layer is the component that directly communicates with the PHY. It is typically a blend of custom hardware and software solutions. [19]

The hardware part usually implements all computationally intensive tasks, such as CRC generation/validation, random number generation, and AES encryption.

The software part of this layer is the only hard real-time part of the whole stack because it has to match the radio hardware's timings based on the PHY layer's specifications.

Multiple State and Role		Advertising	Scanning	Initiating	Connection Master Role	Slave Role
Advertising		Prohibited	Allowed	Allowed	Allowed	Allowed
Scanning		Allowed	Prohibited	Allowed	Allowed	Allowed
Initiating		Allowed	Allowed	Prohibited	Allowed	Prohibited
Connection	Master Role	Allowed	Allowed	Allowed	Allowed	Prohibited
	Slave Role	Allowed	Allowed	Prohibited	Prohibited	Prohibited

Table 2.1: Link Layer State Machine limitations (obsolate)

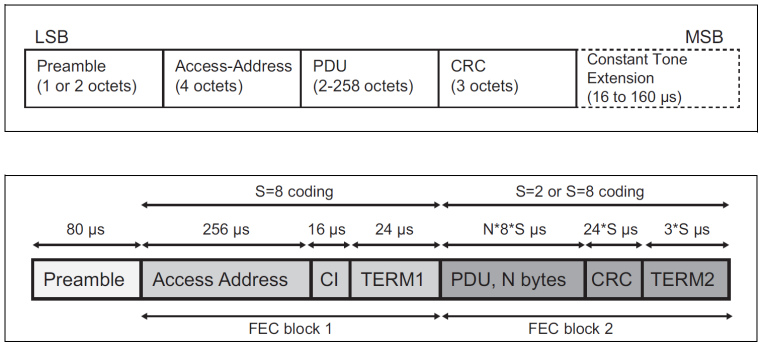

Figure 2.3: AIP formats. (Source: [9])

The Link Layer can be thought of as a state machine in operation. In Figure 3, the representation of this state machine can be seen. The Link Layer may have multiple instances of this state machine but shall have at least one supporting Advertising or Scanning.[7]

The Core Specification 5.2 introduced two (2) new states: Synchronization and Isochronous Broadcasting.

The Air Interface protocol defines the inner working of each state. This interface consists of multiple access schemes, device discovery, and Link Layer connection methods.

In the case of multiple state-machine instances, the first version of the specification ([7]) prohibited certain combinations of states (and roles). Table 2.1 specifies the allowed and prohibited state machine combinations. From the release of Core Specification 5.2, an implementation may support any combination of states and roles. The only criterion is that any two (2) devices shall not have more than one connection between them. [9]

After the introduction in Core Specification 5, there are two algorithms for channel selection. The initial algorithm (called Channel Selection Algorithm #1) only supports channel selection for connection events. Algorithm #2 can be used for connection and periodic advertising events. J. Yin et al. showed that this update significantly enhanced the robustness of the protocol. [23]

The Link Layer Control protocol (LLCP) is used to control and negotiate aspects of the operation of a connection between two Link Layers. This includes procedures for control of the connection, starting and pausing encryption, and other link procedures. [7]

One of the most significant procedures is the Channel Map Update Procedure. This ensures that upon a physical channel update (initiated by frequency hopping), both the master and the slave stay in sync with the physical channel index.

BLE devices communicate via air interface packets. The specification differentiates two different formats; uncoded and coded packets. Figure 2.3 shows the differences between

the formats. Higher-level protocols are transmitted in the PDU field. As Figure 2.3 shows, the largest data transmitted in a single packet is limited to 258 bytes.

2.2.4 Host Controller Interface

The Host Controller Interface (HCI) connects the Controller (low-level) and Host (high-level) parts of the BLE stack. It contains a technology-independent logical command interface structure (HCI driver) and a Host Controller Transport Layer. The HCI provides a uniform interface for accessing a Bluetooth Controller's capability.

According to the standard, separating was necessary to keep the HCI driver independent of the underlying transport technology. Currently, in the latest standard (Core Specification 5.4), there are four (4) different technologies defined for the transport layer: UART, USB, Secure Digital (SD), and Three-wire UART.

The HCI uses a command-response communication format. The Host sends commands to the Controller, and the Controller responds in the form of events. If the Controller cannot execute the command immediately, it shall notify the Host that it received it.

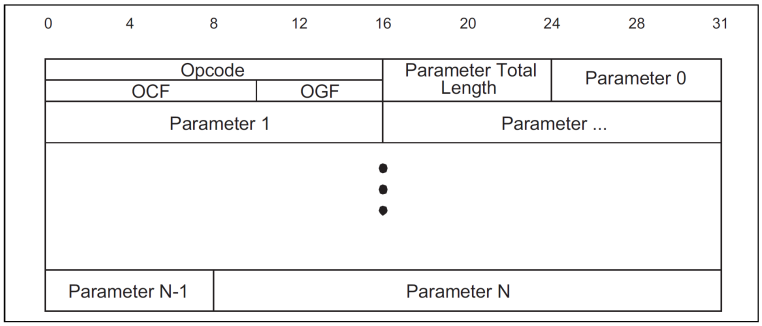

Figure 2.4: HCI command message structure. (Source: [8])

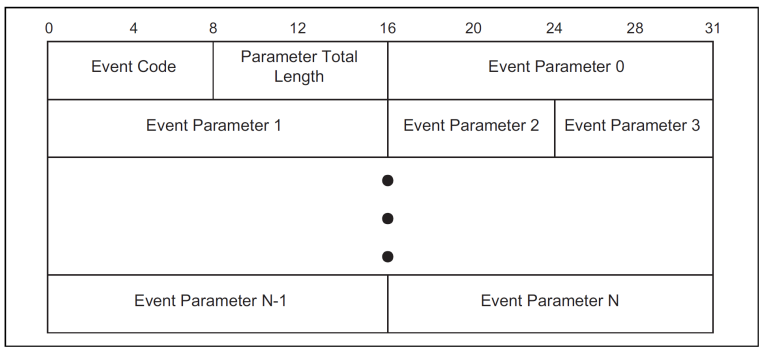

Figure 2.5: HCI command message structure. (Source: [8])

Figure 2.4 and Figure 2.5 show the data formats used in HCI communication.

2.2.5 Host

2.2.5.1 Logical Link Control and Adaption Protocol

The Bluetooth logical link control and adaptation protocol (L2CAP) supports higher-level protocol multiplexing, packet segmentation and reassembly, and the conveying of quality of service information.

Packet segmentation is necessary because data must fit into 27-byte maximum payload size of the BLE packets on the transmit side. [19] On the reception path, this procedure has to be reversed.

The Bluetooth logical link control and adaptation protocol (L2CAP) supports higher-level protocol multiplexing, packet segmentation and reassembly, and the conveying of quality of service information. The protocol state machine, packet format, and composition are described in this document. [19]

2.2.5.2 Attribute Protocol

The Attribute Protocol (ATT) defines the peer-to-peer communication scheme. It defines client and server roles during communication. The ATT client communicates to an ATT server on a remote device. This communication uses a dedicated fixed L2CAP channel created by the initial connection.

The client can send commands, requests, and confirmations to a server. On the reversed path, the server can only send responses, notifications, and indications back to a client, meaning that a client can read and write the values of attributes on a device with an ATT server.

2.2.5.3 Generic Access Profile

The Generic Access Profile (GAP) defines common functionalities all Bluetooth devices use. These functionalities, such as modes and access procedures, are used by transports, protocols, and application profiles. The GAP includes device discovery, connection modes, security, authentication, association models, and service discovery.

2.2.5.4 Generic Attribute Profile

The Generic Attribute Profile (GATT) builds on the Attribute Protocol (ATT) and adds a hierarchy and data abstraction model on top of it. It can be considered the backbone of BLE data transfer because it defines how data is organized and exchanged between applications.

The GATT also provides an interface for discovering, reading, writing, and indicating service characteristics and attributes.

The Bluetooth SIG specifies over 250 standard profiles that must be supported to facilitate interoperability between developers. [10]

2.3 IEEE 802.15.4

The IEEE 802.15.4 standard defines the physical layer (PHY) and medium access control (MAC) sublayer specifications for low-rate wireless personal area networks (LR-WPANs). [15] LR-WPANs are designed to support low-power communication between devices with limited resources and short-range communication requirements.

The standard provides a framework for building wireless networks with low data rates, low power consumption, and low cost. It operates in the unlicensed Industrial, Scientific, and Medical (ISM) bands, such as the 2.4 GHz band used by Wi-Fi and Bluetooth devices, as well as the 868 MHz and 915 MHz bands. It supports a maximum data rate of 250 kbps.

2.3.1 Physical layer (PHY)

2.3.2 Medium Access Control Layer (MAC)

The MAC layer describes essential message handling and congestion control. This layer includes mechanisms for forming and joining a network, a CSMA-CA mechanism, and a link layer for retires and acknowledgment for reliable communication. (ug103-02)

For device identification, the layer uses 16-bit short addresses and 64-bit extended addresses. Devices can be assigned a unique short address by the network coordinator to enable efficient addressing in the network.

In MAC, two roles are defined: the network coordinator and the device. The network coordinator initiates and manages the network, while devices can join the network and communicate with the coordinator or other devices.

2.4 Zigbee

2.4.1 Introduction

Zigbee is a wireless communication protocol specifically designed for applications requiring low-power consumption and short-range communication. It operates on the IEEE 802.15.4 standard and provides a reliable and efficient way for devices to connect and communicate with each other. Zigbee is a frequently used technology in home automation, industrial control systems, and various other applications related to the Internet of Things (IoT). It supports mesh networking, allowing devices to relay data to extend the network coverage. Zigbee is known for its low power consumption, simple implementation, and ability to support a large number of devices in a network.

The Zigbee Alliance created the first version of the specification back in 2004. In 2022, the alliance renamed itself to Connectivity Standards Alliance. In 2023, more than 400 companies are actively involved in the alliance. [1]

The first Zigbee specification was released in 2004. There have been three iterations of the specification, namely Zigbee-2004 (now deprecated), Zigbee-2006, and Zigbee-2007. In 2023, the latest release is Zigbee-2007 Revision 23.

The Zigbee-2007 specification introduced a new stack called ZigBee Pro. The main difference is in the network layer. The ZigBee stack is formed around a central coordinator and uses tree addressing to establish a network. Meanwhile, the Pro version uses stochastic addressing to avoid limitations with a tree. [16]

The Pro stack implements new features in the routing protocol, such as network-level multicast, many-to-one, and source routing. Other features are fragmentation, asymmetric link handling, frequency agility, PAN ID conflict resolution, and multiple security levels. Silicon Labs states that using a Pro network improves stability and reliability.

Both the Pro and the original stack can interoperate but with limitations. If a Pro coordinator forms a network, the network can only contain Pro routers. Any stack can act as an end device. [16]

This thesis will be based on ZigBee Pro Revision 21. This version of the specification is also called ZigBee 3.0.

ZigBee 3.0 contains a new document called Base Device Behavior Specification (BDB). It possesses mandatory and optional application layer behavior for any ZigBee product. [14], [18]

2.4.2 Network

A Zigbee network is made up of three different types of nodes: Coordinator, Router, and End Device. [21]

A Zigbee network can be configured to support different topologies: star, tree, and mesh. Figure 2.6 shows the difference between the topologies.

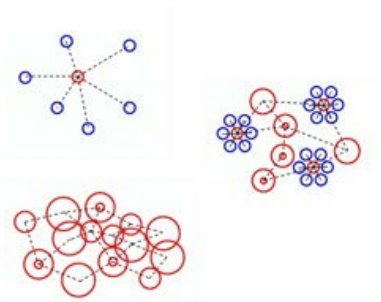

Figure 2.6: Star, Full Mesh, and Hybrid Mesh topologies. Source([16])

2.4.3 Architecture

Zigbee builds on top of the IEEE 802.15.4 physical layer defined in Section 3. The Zigbee architecture defines two more layers: the network layer (NWK) and the application layer (APL).

2.4.4 Network layer

The NWK layer is responsible for mesh networking, which includes broadcasting packets across the network, determining routes for unicasting packets, and ensuring packets are sent reliably from one node to another. The network layer also has a set of commands for security purposes, including secure joining and rejoining. ZigBee networks are all secured at the NWK layer, and the entire payload of the NWK frame is encrypted

2.4.5 Application layer

The ZigBee application layer consists of the APS sublayer, the ZDO, and the defined by manufacturer application objects. (Wang et al., 2016) [5]

2.4.5.1 Application Support Sub-layer

The application support sub-layer (APS) provides an interface between the NWK and the application layer through services for ZDO and application objects.

The APS can be divided into two services: data service and management service. The data service is APS Data Entity (APSDE), and the management service is APS Management Entity (APSME).

The APSDE is responsible for payload conversion, address filtering, reliability, and fragmentation. Once a connection is built up, it is the transport layer between two or more devices.

The other service, the APSME, allows an application to interact with the stack. It is responsible for AIB, binding, security, and group management.

2.4.5.2 Application Framework

The application Framework provides a hosting environment for application objects on a Zigbee device. A developer can define up to 240 distinct application objects, each identified by an endpoint address from 1 to 240.

Index 0 is reserved for communication with ZDO, and index 255 is considered the broadcast address. For future use, the alliance reserved the index range from 241 to 254.

2.4.5.3 Zigbee Device Object

Zigbee device object interfaces the application objects, the device profile, and the APS. For the application objects, it provides device control and network functions. Other functions are address management of the device, discovery, binding, and security. (Wang et al., 2016)

The ZDO is responsible for initializing the APS sublayer, the NWK layer, and the Security Service Provider. Also, the ZDO defines the role of the device within a ZigBee network.

2.4.6 Cluster Library

2.5 Thread/OpenThread

2.5.1 Introduction

Thread is intended to be an open standard, and the protocol is built by incorporating many current and proposed standards.[20] Anyone can download the complete specification from the Thread Group website. [12]

As mentioned, the Thread Group develops and maintains the protocol. Thread Group is a not-for-profit organization formed by seven companies: ARM (Softbank), Big Ass Fans, Freescale (NXP), Nest Labs (Google), Samsung, Silicon Labs, and Yale Locks. More than 200 companies, such as Apple, Google, and Amazon, are forming the Thread Group. [13]

This thesis uses the Thread specification, v1.3.0 2023.

2.5.2 Private Area Network

In a Thread network, there are two types of devices (nodes): Full Thread Device (FTD) and Minimal Thread Device (MTD). FTDs can communicate with each other and with their attached MTD children. MTDs can only communicate with the parent FTD.

A Thread Device can act in six different roles.

1. Active Router

2. Router.Eligible End Device (REED)
3. Full End Device (FED)
4. Minimal End Device (MED)
5. Sleepy End Device (SED)
6. Synchronized Sleepy End Device (SEED)

In order to create a network, the initial node must select an IEEE 802.15.4 channel and a PAN Identifier (PAN ID). The forming node becomes the PAN's first Router and selects a Rouder ID for itself.

The topology is based on the number of Routers in the Thread Network. If there is only one Router, then a basic star topology with a single Router is formed. A mesh topology is automatically formed if there is more than one Router. Figure 2.7 shows a basic topology.

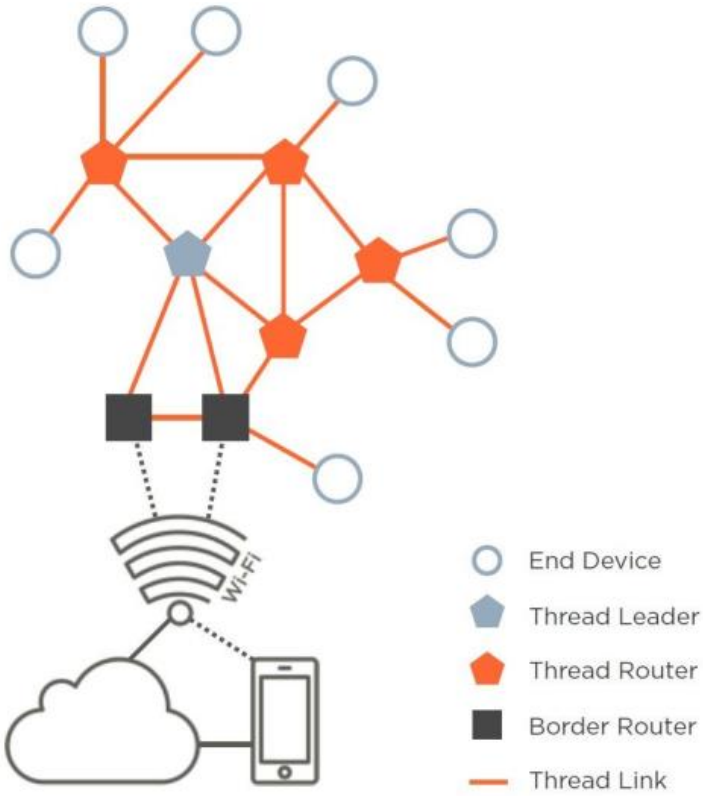

Figure 2.7: Channel allocation. (Source: Figure 3 [11])

The Mesh Commissioning Protocol (MshCoP) manages how a new device can join a Thread Network. A commissioner device is an elected authentication server. This device can be a cell phone, a cloud service, or an FTD. There can only be one commissioner node in a PAN, and the Thread Management Protocol manages it.

A new device that wants to join the network is called Joiner. A Joiner Router is a node that can act as a Router and is one hop away from the Joiner.

There are two main approaches to commissioning: *external* if the commissioner is connected to the Thread Network via a Border Router or *native* if the commissioner is in the Thread Network. Different approaches are shown in Figure 2.8.

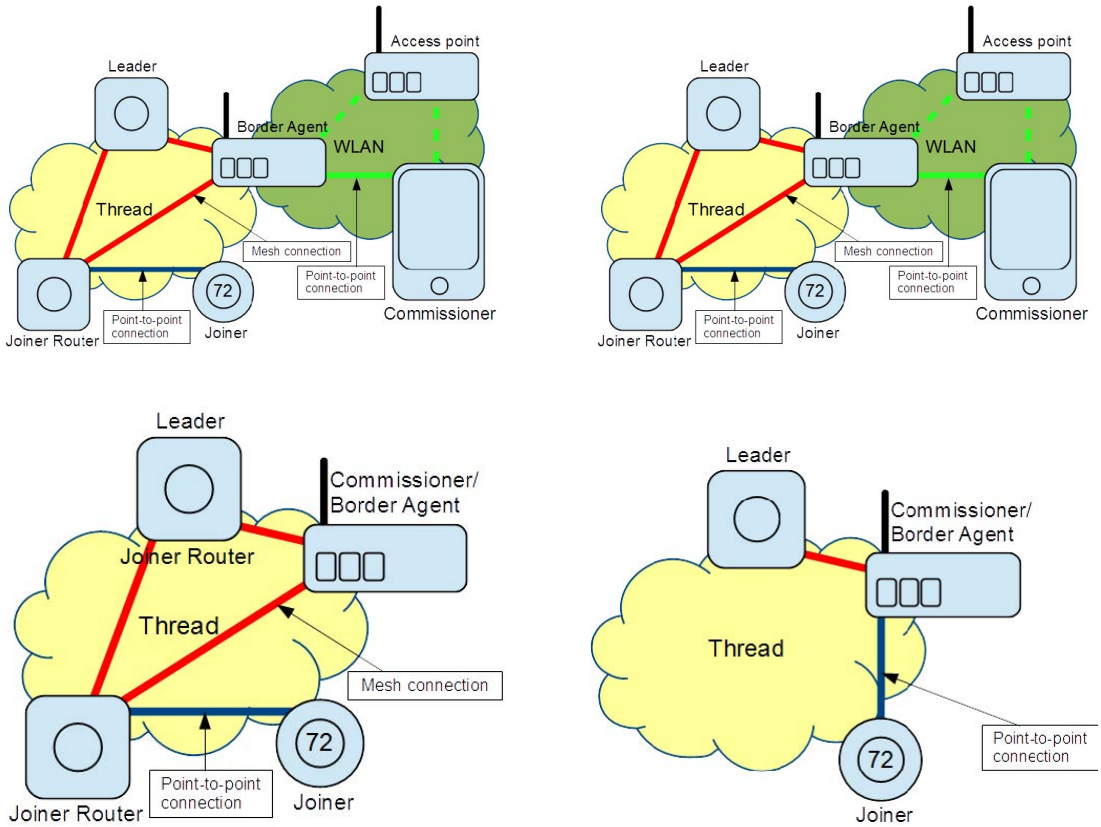

Figure 2.8: External and Nativ commissioning. (Source: [12])

State machines can define the behavior of the Joiner (see Figure 2.9), the Joiner Router (see Figure 2.10), and the Commissioner (see Figure 2.11).

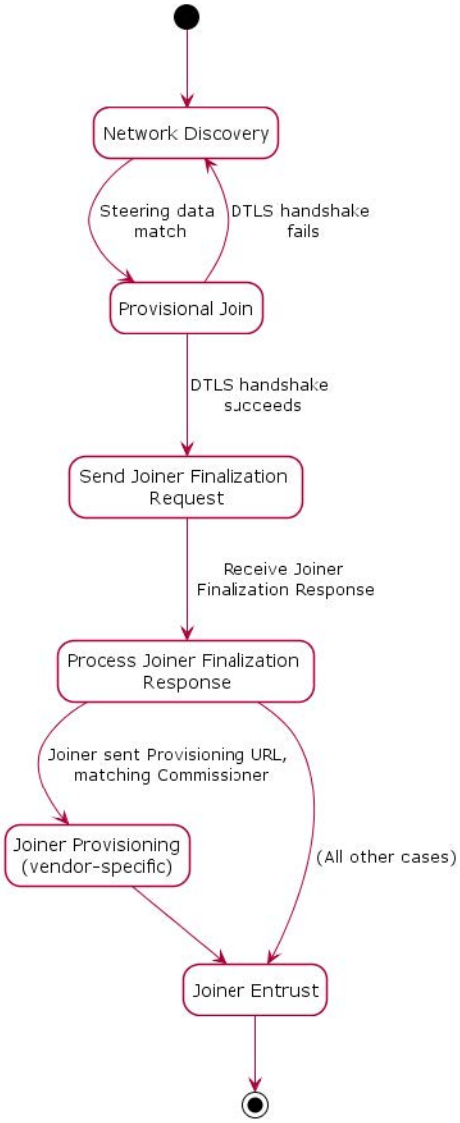

Figure 2.9: The Joiner's state machine (Source: [12])

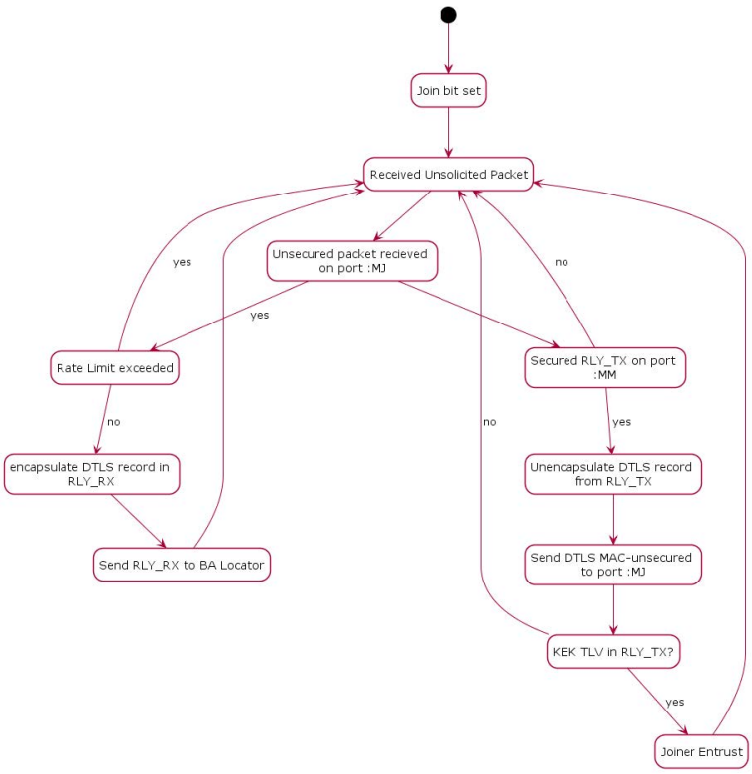

Figure 2.10: The Joiner Router's state machine (Source: [12])

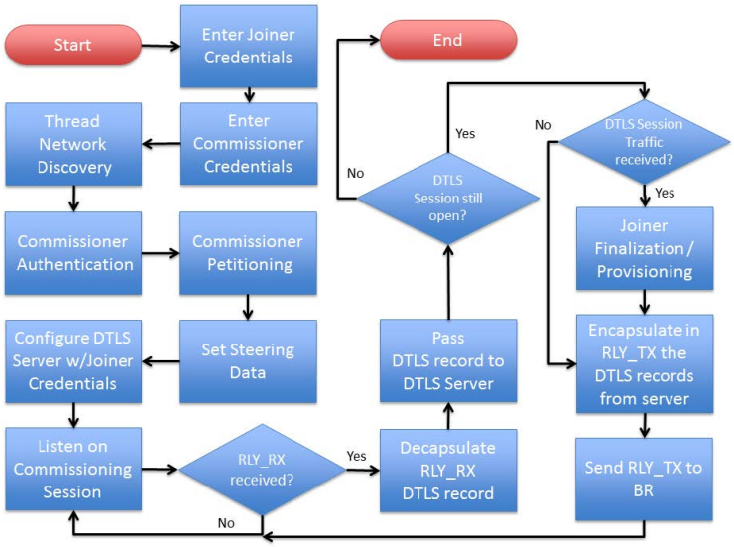

Figure 2.11: The Commissioner's state machine (Source: [12])

Chapter 3

Multi-Protocol

In this chapeter I will define the multi-protocol networking in IoT technologies.

Section 3.1 will give a brief overview of the technology. Section 3.2 will showcase some configurations and Section 3.3 will introduce some implementations that are available.

3.1 Overview

As the IoT market continues to evolve and grow, developers need more integrated options for wireless connectivity. (NXP, website) With multiprotocol-enabled wireless solutions reduce design and development costs and open the path for IoT convergence where applications can leverage the best aspects of multiple connectivity standards.

As the current generation of chips on the market, a multiprotocol implementation usually contains some kind of operating system or a simple scheduler to manage concurrency between the protocols.

Within multiprotocol, the industry specifies different types. In a simple switched multiprotocol, the device's bootloader can switch between two or more protocols. This method does not require an operating system, as the switching is not that dynamic. In switched mode, only one protocol can be loaded at any given time during operation. Figure 3 shows the operation of switched mode.

The other main type is concurrent multiprotocol. This mode allows the device to run multiple protocols on a single radio. The primary features of this type of solution are: Switch between applications Manage and configure the physical layer Protocol prioritization Real-time scheduling

These features usually call for an operating system. Available from the market, manufacturers do not create this operating system from the ground up. Rather they create a specialized Real-Time Operating System (RTOS).

Some manufacturers offer Linux-based solutions as well as embedded OS. With more powerful and capable embedded computers emerging on the market, devices can run more general operating systems such as Linux. These devices have the capacity to run multiple protocols alongside the OS and an additional IP stack as well. Devices suitable for this workload are being used as gateways or translators between protocols in the IoT realm.

3.2 Device configuration

As I have introduces the main protocols in Chapter 2, I have not mentioned what configurations these can be used. All stacks have a clear separation of what each layer does. This creates a possibility of different manufacturers can implement different layers.

Most manufacturers extend beyond their System-on-Chip (SoC) design and allow more interfaces so their network device to improve reusability. Figure 4 shows different approaches used in the market.

The SoC approach provides all stack-functionality and application-layer components within a single chip. With SoC, developers can write the most optimized solutions with the lowest power consumption. But the whole application is constrained to the actual hardware limitations.

In a Network Co-Processor (NCP) approach, the chip only runs the functions necessary for networking. The most common configuration is that the chip communicates with a second device called the "host" processor via either SPI or UART. The host is responsible for the application management.

A Radio Co-Processor (RCP) approach is similar to the NCP. The chip only runs the physical layers of the stacks while the host runs the application and network layers. With this architecture integration of multiple protocols that use the same radio band can be better achieved.

3.3 Implementations

3.3.1 Dynamic Multi-protocol Manager - Texas Instruments

3.3.2 Dynamic Multiprotocol - Silicon Labs

3.3.3 CPCd - Silicon Labs

3.3.4 Multi-Protocol Service Layer - Nordic Semiconductor

Chapter 4

Software testing

4.1 Overview

A lot of different standard bodies and writers have defined software testing. The International Software Testing Qualifications Board (ISTQB) defines software testing as follows: ”The process consisting of all lifecycle activities, both static and dynamic, concerned with planning, preparation, and evaluation of software products and related work products; to determine that they satisfy specified requirements, to demonstrate that they are fit for purpose, and to detect defects.” [4]

Testing can have different goals, such as getting information about the quality, supporting decision-making, detecting defects, or preventing defects. In the process, there are two different approaches: test-as-information-provider and test-as-quality-accelerant.

In the test-as-information-provider approach, testing is usually executed last in development. There are independent test teams and separate test phases with fixed release cycles. Meanwhile, test-as-quality-accelerant follows a test-always strategy. In the latter approach, testers are quality assistants, and developers write tests alongside testers. This allows shorter and more fine-grade releases.

4.2 Testing levels

According to ISTQB, test levels are groups of test activities that are organized and managed together. Each test level is an instance of the test process performed concerning software at a given stage of development, from individual components to complete systems or, where applicable, system of systems. [4]

The ISTQB differentiates the following levels: component (unit), component integration (integration), system, system integration, and acceptance testing. Unit testing focuses on a single component in isolation and is usually performed by developers. Integration testing focuses on testing the interfaces and interactions between components. System tests focus on the overall behavior and capabilities of the entire system or product.

Whittaker et al. show [22] a different approach to test levels used in Google. Test levels are defined in execution time; see Table 4.1. Google uses three different levels: small, medium, and large tests. Small tests cover a single unit of code in a faked environment. Medium tests cover two or more interacting features. Large tests represent real user scenarios and use real user data sources.

	Small tests	Medium tests	Large tests
Time Goals	$\leq$ 100 ms	$\leq$ 1 sec	As quickly as possible
Time Limits	Kill target after 1 minute	Kill target after 5 minutes	Kill target after 1 hour

Table 4.1: Google's test levels. (Source: [22])

4.3 Test types

Test types are groups of test activities related to specific quality characteristics, and most of those test activities can be performed at every test level. [4]

Common types used in the industry are functional, non-functional, and regression testing.

Functional testing evaluates the functions that a system (or component) should perform. The main objective is to check the functional completeness and correctness of the system.

In Non-functional testing, the goals are to test how well the system behaves. Commonly used software quality characteristics are defined by ISO/IEC 25010 standard. These characteristics are performance, compatibility, usability, reliability, security, maintainability, and portability. Non-functional tests may need specific test environments.

Regression testing confirms that changes in the codebase did not inject new defects into the functionality. According to ISTQB, performing an impact analysis is advisable to optimize the extent of the regression testing.

4.4 Model-based test generation

4.5 Runtime verification

4.6 Bluetooth testing

4.7 Zigbee testing

4.8 Thread/Openthread testing

Acknowledgements

Ez nem kötelező, akár törölhető is. Ha a szerző szükségét érzi, itt lehet köszönetet nyilvánítani azoknak, akik hozzájárultak munkájukkal ahhoz, hogy a hallgató a szakdolgozatban vagy diplomamunkában leírt feladatokat sikeresen elvégezze. A konzulensnek való köszönetnyilvánítás sem kötelező, a konzulensnek hivatalosan is dolga, hogy a hallgatót konzultálja.

Bibliography

[1] Csa members. https://web.archive.org/web/20230526073333/https://csa-iot.org/members/, 2023.

[2] Bluetooth Special Intrest Group. Our history (2018. may 25.). https://web.archive.org/web/20180525083558/https://www.bluetooth.com/about-us/our-history, 2018.

[3] Bluetooth Special Intrest Group. Specifications and documents (2023. 05. 16). https://www.bluetooth.com/specifications/specs/?types=specs-docs&keyword=Core+Specification, 2023.

[4] International Software Testing Qualifications Board. Certified tester foundation level syllabus v4.0. https://istqb-main-web-prod.s3.amazonaws.com/media/documents/ISTQB_CTFL_Syllabus-v4.0.pdf, 2023.

[5] NXP B.V. *ZigBee 3.0 - Facilitating the Internet of Things*, 2016.

[6] Myra Dideles. Bluetooth: A technical overview. *XRDS*, 9(4):11–18, jun 2003. ISSN 1528-4972. DOI: 10.1145/904080.904083. URL https://doi.org/10.1145/904080.904083.

[7] Bluetooth Special Intrest Group. Bluetooth core specification 4.0. Technical report, 2010.

[8] Bluetooth Special Intrest Group. Bluetooth core specification v5.0. Technical report, 2016. https://www.bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=421043.

[9] Bluetooth Special Intrest Group. Bluetooth core specification v5.2. Technical report, 2019. https://www.bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=421043.

[10] Bluetooth Special Intrest Group. Gatt specification supplement 9. Technical report, 2023. https://www.bluetooth.org/docman/handlers/DownloadDoc.ashx?doc_id=569770.

[11] Thread Group. *Thread Network Fundamentals White Paper*, 2022. https://portal.threadgroup.org/DesktopModules/Inventures_Document/FileDownload.aspx?ContentID=633.

[12] Thread Group. Thread 1.3.0 public specification. Technical report, 2023. https://www.threadgroup.org/ThreadSpec.

[13] Thread Group. Thread group - members. https://web.archive.org/web/20230525142814/https://www.threadgroup.org/thread-group, 2023.

[14] Texas Instruments. *What's new in Zigbee 3.0*, 2019.

[15] Adam Kozłowski and Janusz Sosnowski. Analysing efficiency of ipv6 packet transmission over 6lowpan network. page 104451J, 08 2017. DOI: 10.1117/12.2280699.

[16] Silicon Labs. *UG103.2: Zigbee Fundamentals*, 2023. https://www.silabs.com/documents/public/user-guides/ug103-02-fundamentals-zigbee.pdf.

[17] Martin Leopold, Mads Bondo Dydensborg, and Philippe Bonnet. Bluetooth and sensor networks: A reality check. In *Proceedings of the 1st International Conference on Embedded Networked Sensor Systems*, SenSys '03, page 103–113, New York, NY, USA, 2003. Association for Computing Machinery. ISBN 1581137079. DOI: 10.1145/958491.958504. URL https://doi.org/10.1145/958491.958504.

[18] Philipp Morgner, Stephan Mattejat, Zinaida Benenson, Christian Müller, and Frederik Armknecht. Insecure to the touch: Attacking zigbee 3.0 via touchlink commissioning. In *Proceedings of the 10th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, WiSec '17, page 230–240, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450350846. DOI: 10.1145/3098243.3098254. URL https://doi.org/10.1145/3098243.3098254.

[19] Kevin Townsend, Carles Cufi, Akiba, and Robert Davidson. *Getting Started with Bluetooth Low Energy*. O'Reilly Media, Inc., 2014.

[20] Ishaq Unwala, Zafar Taqvi, and Jiang Lu. Thread: An iot protocol. In *2018 IEEE Green Technologies Conference (GreenTech)*, pages 161–167, 2018. DOI: 10.1109/GreenTech.2018.00037.

[21] Lindsey N. Whitehurst, Todd R. Andel, and J. Todd McDonald. Exploring security in zigbee networks. In *Proceedings of the 9th Annual Cyber and Information Security Research Conference*, CISR '14, page 25–28, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450328128. DOI: 10.1145/2602087.2602090. URL https://doi.org/10.1145/2602087.2602090.

[22] James A. Whittaker, Jason Arbon, and Jeff Carollo. *How Google Tests Software*. Addison-Wesley Professional, 2012.

[23] Junjie Yin, Zheng Yang, Hao Cao, Tongtong Liu, Zimu Zhou, and Chenshu Wu. A survey on bluetooth 5.0 and mesh: New milestones of iot. *ACM Trans. Sen. Netw.*, 15(3), may 2019. ISSN 1550-4859. DOI: 10.1145/3317687. URL https://doi.org/10.1145/3317687.